9. El plazo de entrega se divide en dos momentos:
a. El día 09 de octubre para adjuntar la evidencia del proceso de análisis y diseño de la solución (Diagrama de clase incluido).

Contextualización: Para esta actividad se pide simular una versión inicial de una plataforma social con funcionalidades para agregar amigos, enviar solicitudes de amistad, reaccionar a fotos de otras personas, etc… Para ello se pide utilizar lenguaje C++, punteros, memoria dinámica, y programación orientada a objetos; procurando siempre escoger las variables y el diseño adecuado para cumplir con todas las funcionalidades que esta versión de la plataforma tendrá y sentar algunas bases para la próxima.

Análisis: Para este desafío deberemos leer y escribir en archivos, sin embargo leerlos completos puede resultar muy costoso y a veces innecesario. Se implementará una estrategia donde se separe la información que requiere cada sección del programa en archivos diferentes. Ya que en ocasiones no toda la información se necesitará que se cargue para la acción específica que requiere el usuario, pero, en caso de haberse cargado ya, se quedará así. Solo se liberará la memoría cuando hayan cambios en los datos (teniendo ahora un arreglo que contenga toda nueva información) que requieran ser guardados en ese instante. Un ejemplo es, el usuario inicia sesión, quiere aceptar o enviar una solicitud de amistad: en este punto, es innecesario para el programa cargar los archivos de mensajes, ya que esta información no es requerida por el momento para la acción que el usuario quiere realizar.

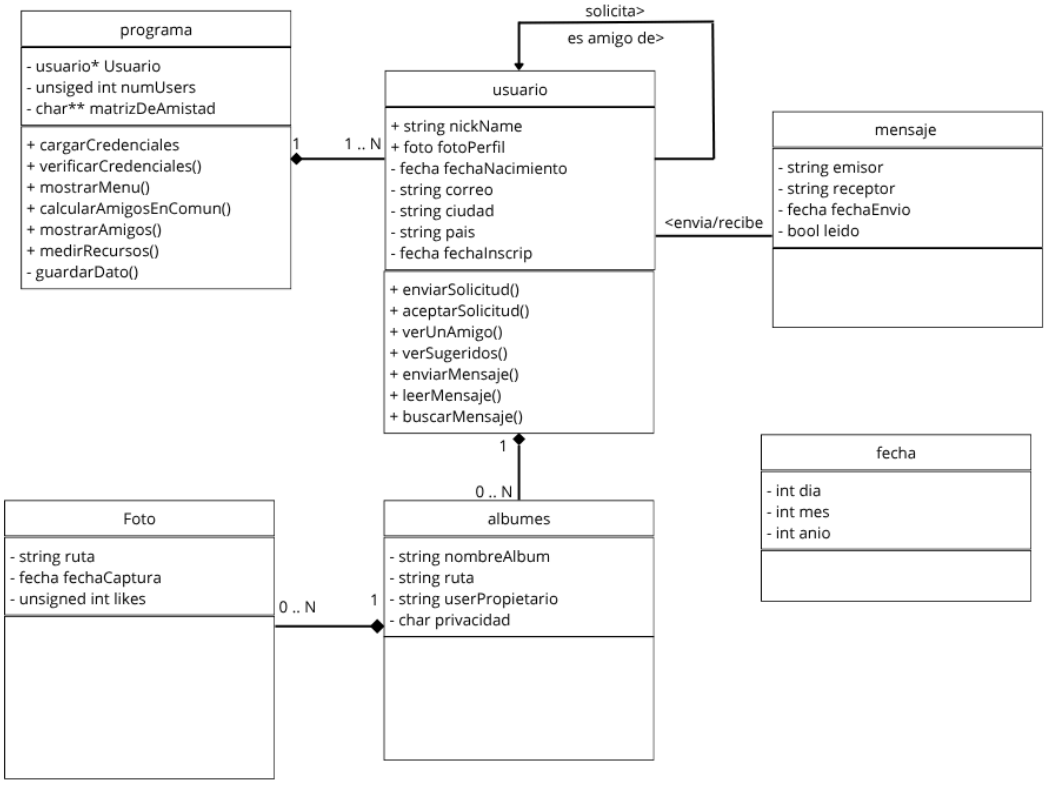
Los datos que cada usuario tendrá son: nombre de usuario, fecha de nacimiento, correo, ruta de su foto de perfil, ciudad, país, hora de inscripción a la plataforma, una lista de 0 a n amigos, una lista de mensajes que ha enviado, una lista de mensajes que ha recibido y una lista de solicitudes de amistad.

Las sugerencias de amistad se pueden calcular durante la ejecución para ese usuario específico en base a los amigos en común (uso de la matriz de adyacencia). Los álbumes y las fotos son clases aparte.

Consideraciones: Para la matriz de adyacencia que indica quien es amigo de quien, se entiende que es una matriz simétrica con ceros en su diagonal (estos ceros no aportan información útil) y unos en donde haya una relación de amigos (esto se sabrá al leer la lista de amigos de cada usuario), por lo cual se puede pensar en solo tener en cuenta la sección triangular superior de esta matriz, lo cual nos ahorra procesos innecesarios para generar y leer todos los datos de esta matriz, además de consumir un 50% menos de memoria de lo que consumiría la matriz original.

En el desarrollo del proyecto se intentará minimizar al máximo la cantidad de memoria usada. Por ejemplo, representando la matriz anterior no como una matriz de chars, si no usando operaciones a nivel de bits, así como en el desafío 1, donde cada 3 bits representaba un estado.

Propuesta:



CONSIDERACIONES Y NOTAS QUE TUVE EN CUENTA PARA ESTE DESAFÍO (informalmente)

En el futuro se espera que Ude@Book opere online a nivel mundial

Cada usuario tiene asociado: Nombre de usuario único, fecha de nacimiento, correo, foto de perfil (ruta), contraseña, ciudad, país y una fecha de inscripción en la plataforma. También tiene asociados conjuntos de: amigos, mensajes enviados, mensajes recibidos, solicitudes, sugerencias, álbumes y fotos.

Cero o más amigos. Los amigos son otros usuarios registrados en la plataforma con los que se ha definido mutuamente una relación de amistad.
Un conjunto de mensajes que ha enviado, ordenados por fecha de envío. Cada mensaje enviado puede tener dos estados: leído o no leído.
Un conjunto de mensajes que ha recibido, ordenados por fecha de envío.
Un conjunto de solicitudes de amistad, las cuales puede aceptar o rechazar.
Un conjunto de sugerencias de amistad. Las sugerencias de amistad muestran a los amigos de sus amigos directos, y se rankean según la cantidad de amigos en común.

Cero o más álbumes, los cuales tienen un nombre, una ruta al almacenamiento y una configuración de privacidad, pudiendo ser: públicos (puede verlos cualquier usuario de la plataforma), privados (sólo puede verlos el dueño), o amigos: sólo para amigos. Adicionalmente, cada álbum contiene cero o más fotos.
De cada foto se conoce: La fecha de su captura, tamaño, ruta al archivo en el almacenamiento y conteo de likes. Cada foto pertenece a un único álbum. formato .png, Por ejemplo, si el usuario juanito76 tiene el álbum “Cartagena2026” y en él las fotos “Malecon2612” y “Bocagrande1”, al perfil de dicho usuario se asociarán las rutas:

/users/(sub-ruta-X-fija)/juanito76/albums/Cartagena2026/Malecon2612.png
/users/(sub-ruta-X-fija)/juanito76/albums/Cartagena2026/Bocagrande1.png
/users/(sub-ruta-X-fija)/juanito76/albums/profile.png

*La última ruta corresponde al almacenamiento de la foto de perfil del usuario.
**La sub-ruta-X-fija es la misma para todos los usuarios de la plataforma.

1- Explicar la elección del formato de mis archivos en el informe. No todos los datos se actualizan.
2- Verificar credenciales. (pre-menu). Menú (opciones y besties(calc # amig en comn de Amigos (priori pa ver) y actlza (ver otros 3 besties))).
3- a. solictAmist() (hay q saber el userName y validar). b. List solicit, luego aceptarSolicitud(userName)
4- Ver amigos. verPerfil (string& userName const). Ver albumes, next fotos (depend priv, carrusel). 4 interacciones (salir del álbum, sgte, previa, dar/quitar like a la foto (condicional pa dar o quitar like) (no se requiere guardar si ya le habiamos dado like antes, siempre aumentado los likes))
5- verSugeri2(short k) (calc # amig en comn de noAmigos (orden de + a -)). Opción enviarSolicitud(userName de un Sugerido)
6- sendMssg(Amigo) (valid, # chars (512)). verNewMssgs() (solo 1 user (opcional)) then, mensaje leido. searchMssgs(fchInicial, fchFinal), then verMensajesRecibidos() pertenecientes a [fchInicial,fchFinal].
7- Luego de terminar de ejecutar any de 1-7: medirIteraciones() y memoUsada(). Mencionar al final si se usó alguna funcionalidad de una libreria externa, como la del string con un .append.

------ Considerar cuando es necesario crear la matriz de amigos y solicitudes.
------ Considere las restricciones implícitas en la lógica del problema. Por ejemplo, no enviar solicitudes a quienes ya son amigos o a quienes actualmente nos han solicitado amistad, no enviar mensajes vacíos, entre otras.

Usar constructores de copia, getters, setters, sobrecarga de métodos y de operadores (sobrecargar al menos '=' y '=='), etc.

Tenga presente el criterio de eficiencia, es clave pa la calificación de su proyecto.

Consideraciones:

Si el usuario tuviera solo entre 0 y 3 amigos, que fotos de perfil vería o que ruta? R/ Se imprimirá un mensaje bonito para decirle que no hay nada que ver. Se refiere a los amigos de él.
La visualización aleatoria de los perfiles de los amigos solo podrá ser actualizada una vez (recuerde)

Duda en Leer nuevos mensajes: Dado el nombre de un usuario, se visualizan los mensajes enviados por este? en caso de que se visualicen todos, debería ordenarse por hora de envio o por usuario? (pienso que usuario sería lo mejor y que debería decirse antes de quienes tenemos nuevos mensajes y luego seleccionar el usuario a quien quiero leer los mensajes. Autoriza, parece chevere. Esto implica que a pesar de que se ejecute la función de leer mensaje, no todos los mensajes serán leidos.

Se puede ordenar los mensajes (leer y buscar) por usuario, pero con que esté en orden de fecha

Para enviar los mensajes por ejemplo, se puede usar una libreria que permita obtener la fecha del usuario.

La actualización y leer archivos es altamente costoso.
Costo de acceder a los archivos.. proponer un modelo de actualización de los archivos que sea conveniente (actualizar cuando sea pertinente, no necesariamente cada que haga un cambio cualquiera). Esto se preguntará en la sustentación.
Se puede verificar el envio de un mensaje iniciando sesión del que lo recibió.
Mínimo habrán 1000 usuarios.
Hay 2 formas de manipular archivos:
1. Acceso secuencial,
2. Acceso directo, es capaz de ir a una linea específica y una porción específica. ESTE MÉTODO NO SE PIDE.

MENU: menú que permita acceder de manera independiente a las funcionalidades requeridas en este enunciado.

MATRIZ DE ADYACENCIA: Consideraciones Técnicas
Manipulación de Bits: Dado que no usas bibliotecas externas, deberás implementar las operaciones a nivel de bits (desplazamientos <<, >> y máscaras &, |) dentro de los métodos de acceso de la matriz (ej. setRelacion(idA, idB, estado) y getRelacion(idA, idB)).
Mapeo de Nombres a Índices: Como los usuarios se identifican por su username, la clase Datos o la MatrizAdyacencia necesitarán un mecanismo (un arreglo ordenado o índice) para traducir username a un ID entero (de 0 a N-1) para saber qué fila/columna consultar en la matriz.